



File Systems

A Semester Project

Part—18

This article, which is part of the series on Linux device drivers, introduces the concept of a file system by simulating one in user space.

If you are looking for a semester project, check this out. Take a careful look at the table of contents of the various books on Linux device drivers. You will find that no book lists a chapter on file systems. Not because they are not used or not useful. In fact, there are close to 100 different file systems, as of today, and most of them are possibly in active use. Then why are they not talked about, in general? And are so many file systems required? Which one of them is the best? Let's try answering these queries, one by one.

In reality, there is no 'best' file system and, in fact, there may never ever be one either. This is because particular file systems suit particular sets of requirements. Thus, different requirements ask for different 'best' file systems, leading to so many active file systems and many more to come. So, basically they are not generic enough to be talked about generically; rather, they are more of a research topic. And the reason for them being the toughest of all drivers is simple—generic material on the topic is almost unavailable, the best documentation being the code of various file systems. Hmmm! So that does sound perfect for a semester project, right?

In order to get started on such a research project, a lot of smaller steps have to be taken. The first one is to understand the very concept of a file system. That could be done best by simulating one in user space. Shweta took the ownership of getting this first step done, as follows:

- Use a regular file for a partition, i.e., for the actual data

storage of the file system (hardware space equivalent)

- Design a basic file system structure (kernel space equivalent) and implement the *format/mkfs* command for it, over the regular file
- Provide an interface/shell to type commands and operate on the file system, similar to the usual bash shell. In general, this step is achieved by the shell along with the corresponding file system driver in kernel space. But here, that translation is embodied/simulated into the user space interface itself. (My next article will cover this in detail.)

The default regular file chosen is *.sfsf* (simulating file system file) in the current directory. Starting '.' (dot) is for it to be hidden. The basic file system design mainly contains two structures, the super block that has the information about the file system, and the file entry structure with the details about each file in the file system. Here are Shweta's structures defined in *sfs_ds.h*

```
#ifndef SFS_DS_H
#define SFS_DS_H

#define SIMULA_FS_TYPE 0x13090015 /* Magic Number for our file
system */
#define SIMULA_FS_BLOCK_SIZE 512 /* in bytes */
#define SIMULA_FS_ENTRY_SIZE 64 /* in bytes */
#define SIMULA_FS_DATA_BLOCK_CNT ((SIMULA_FS_ENTRY_SIZE - (16 + 3
* 4)) / 4)
```